ZENITH SCHOOL OF AI / OPEN-SOURCE EVALUATION

DELIVERY LOGISTICS

SmartDispatch

Delivery Partner Assignment & ETA Engine

FORMAT	DURATION	LANGUAGE	DATA
Individual	24-hour individual evaluation	Python 3.10+	Synthetic / offline

Official evaluation repository: https://github.com/sunitazenith07-lang/SmartDispatch-Open-Source-Evaluation

Open the repository in a browser, then read `STUDENT_PROBLEM_STATEMENT.md` and the README before changing code.

1. Scenario

A food-delivery service assigns orders to delivery partners. Partner availability, existing workload, restaurant preparation time and travel estimates affect the delivery ETA. Build the decision logic; do not build a mobile app, map UI or live GPS integration.

This is a simulated challenge inspired by common logistics problems, not any company's internal code.

2. Required behaviour

1. Validate order ID, restaurant ID, destination zone and non-negative preparation/travel times.
2. Consider only active partners below their order capacity. A candidate cannot receive a reliable ETA if any required travel-time entry is missing.
3. Calculate pickup time as current time + partner-to-restaurant travel + remaining workload. Calculate ETA as pickup time + restaurant preparation + restaurant-to-customer travel.
4. Pick the eligible candidate with the lowest ETA. Break ETA ties by partner ID in ascending lexicographic order.
5. Missing travel data must never be treated as zero. If no valid candidate remains, leave the order unassigned and explain why.
6. Process a cancellation event only once so a duplicate cancellation does not release workload/capacity twice.
7. Return deterministic results and handle empty lists, unknown IDs, malformed records and negative time values safely.

3. Setup and first checks

Use Python 3.10 or newer and Git. From the repository root, run the following commands in order:

```
python -m pip install -r requirements-dev.txt
python -m pytest -q
python -m main
```

4. Work to submit

- Read the source and run the starter tests before editing.
- Treat the required behaviour above as the acceptance contract. Investigate failing assertions, locate the root cause and fix the logic instead of hard-coding fixture results.
- Add regression tests and document at least five edge cases in `TESTING.md`.
- Submit one pull request to the original repository containing your code and tests. Do not merge it.
- Include `DISPATCH_NOTES.md` with one worked ETA, one no-eligible-partner example and three edge cases.

5. Optional stretch work

Choose at most one stretch item, and only after the required behaviour is correct. Stretch work is not required for a strong score.

- Batch-assign orders in supplied priority order.
- Return an explanation for each candidate.
- Recalculate ETA after a preparation-time change.

6. Testing and GitHub Actions

Some visible tests are expected to fail on the untouched starter because key behaviours are incomplete. The exact initial test count is not fixed. GitHub Actions runs on pushes and pull-request updates and reports visible test counts. A red check before your fixes is expected; make the visible tests pass without deleting, skipping, weakening or bypassing them. Do not alter the workflow to evade failures.

7. Individual-work policy

AI chatbots, AI coding assistants, generated patches/tests and asking another person to write or debug your solution are prohibited. Ordinary non-AI language/framework documentation is allowed. Do not upload challenge code or data to external services. You must be able to explain your changes and make a small live modification if asked. A code walkthrough can support integrity checks, but no automated detector can prove whether AI was or was not used.

8. Evaluation rubric - 100 marks

Criterion	Marks
Codebase understanding and ETA trace	**15**
Assignment rules and deterministic tie-breaking	**25**
Edge-case safety and capacity consistency	**20**
Tests	**15**
Code quality	**10**
Pull request, notes and live explanation	**15**
Total	**100**

Technical expectation: Marks reward correct reasoning and behaviour that matches this written contract, not line count. A green test suite alone does not guarantee full marks; be ready to explain your choices, test strategy and known limitations.

The output must be deterministic. Missing travel information is unknown, not zero.

This is a simulated startup-style challenge inspired by common product/operations problems. It is not a confidential or official problem statement from any named company. All datasets are artificial.